

PORTFOLIO PROJECT DOCUMENTATION · TECHNICAL REPORT

# AI Study Assistant

Multi-Agent Intelligent Learning System

*A LangGraph-powered adaptive learning platform with personalized quiz generation, automatic evaluation, and persistent performance tracking.*

## DEVELOPER

Sabir Jan

## DATE

June 2026

## LIVE DEMO

[huggingface.co/spaces/Sabir7Jan/  
study-assistant-multi-agent-system](https://huggingface.co/spaces/Sabir7Jan/study-assistant-multi-agent-system)

## STACK

LangGraph · FastAPI · Gemini ·  
SQLite

PRODUCTION  
DEPLOYED

HUGGING FACE  
SPACES

Full-Stack ML Engineering · Multi-Agent Systems ·  
Cloud Deployment

# Table of Contents

---

|                                            |    |
|--------------------------------------------|----|
| 1. Executive Summary .....                 | 3  |
| 2. Introduction and Motivation .....       | 4  |
| 2.1 Problem Statement .....                | 4  |
| 2.2 Project Goals and Target Users .....   | 4  |
| 3. System Architecture .....               | 5  |
| 3.1 Multi-Agent Pipeline .....             | 5  |
| 3.2 Data Flow .....                        | 5  |
| 4. Technology Stack Justification .....    | 6  |
| 5. Feature Breakdown .....                 | 7  |
| 6. API Design .....                        | 10 |
| 7. Deployment .....                        | 11 |
| 8. Challenges and Solutions .....          | 12 |
| 9. Results and Evaluation .....            | 13 |
| 10. Future Work .....                      | 13 |
| 11. Conclusion .....                       | 14 |
| 12. References and Technologies Used ..... | 15 |

# Executive Summary

---

The AI Study Assistant is a production-ready, multi-agent intelligent learning system that delivers personalized educational experiences through a modular architecture combining LangGraph, FastAPI, Streamlit, and Google's Gemini AI. The system addresses a critical gap in self-directed education: the absence of adaptive, AI-driven tutoring that can explain complex topics, generate contextually relevant assessments, evaluate student responses with nuance, and dynamically adjust difficulty based on individual performance trajectories.

Built with a stateful multi-agent orchestration layer powered by LangGraph, the system routes user requests through specialized agent graphs responsible for topic explanation, quiz generation, answer evaluation, and memory recall. A FastAPI backend provides asynchronous RESTful endpoints, while a Streamlit frontend delivers an intuitive, responsive interface. Persistent SQLite storage ensures that quiz history, scores, and performance metrics are retained across sessions.

Deployed via Docker on Hugging Face Spaces, the application demonstrates a complete full-stack machine learning engineering workflow — from architecture design to cloud deployment. The live deployment is accessible at: [huggingface.co/spaces/Sabir7Jan/study-assistant-multi-agent-system](https://huggingface.co/spaces/Sabir7Jan/study-assistant-multi-agent-system)

## 2.1 Problem Statement

Traditional self-study methods suffer from three fundamental limitations. First, students consuming static educational content such as textbooks or video lectures lack the ability to receive immediate, interactive clarification on concepts they find challenging. Second, assessment tools in conventional learning platforms typically offer fixed-difficulty question banks that do not adapt to individual learner proficiency, resulting in either excessive frustration from overly difficult material or insufficient challenge from repetitive easy questions. Third, existing study tools rarely maintain persistent memory of a learner's performance history, preventing the system from building longitudinal understanding of strengths, weaknesses, and optimal difficulty calibration.

These limitations translate into measurable learning inefficiencies: students spend disproportionate time on already-mastered material, receive delayed feedback on misconceptions, and experience motivation decay due to poorly calibrated challenge levels. The AI Study Assistant was conceived to address each of these gaps through a unified multi-agent architecture that combines natural language understanding, generative AI, and adaptive difficulty algorithms.

## 2.2 Project Goals and Target Users

The primary goal of this project was to build an intelligent, autonomous tutoring system capable of delivering personalized learning experiences at scale. The system was designed for self-directed learners, students preparing for examinations, and educators seeking AI-assisted assessment tools.

The architecture prioritizes three core objectives:

- 1 Natural, conversational topic explanations generated by a large language model
- 2 Dynamically generated quizzes with automatic grading and personalized feedback
- 3 Persistent performance tracking with adaptive difficulty recommendation based on historical accuracy patterns

## 3.1 Multi-Agent Pipeline

The system is organized around a modular multi-agent pipeline that separates concerns across natural language understanding, intent routing, agent graph execution, and persistent storage. The architecture follows a layered design pattern with four principal components:

- ▶ **Frontend Presentation Layer** — Streamlit user interface
- ▶ **API Gateway Layer** — FastAPI RESTful backend
- ▶ **Agent Orchestration Layer** — LangGraph multi-agent graphs
- ▶ **Persistence Layer** — SQLite database

The Natural Language Understanding (NLU) module ( `nlu.py` ) serves as the system's intent classification engine. When a user submits a request, the NLU module parses the input to extract three critical parameters: **intent** (explain, quiz, evaluate, recall, or full), **topic** (the subject matter), and **difficulty** (easy, medium, or hard). This structured representation enables deterministic routing to the appropriate LangGraph agent graph.

The LangGraph agent layer ( `langgraph_graph.py` ) implements five specialized agent graphs, each optimized for a specific educational task. LangGraph was selected because it provides stateful, cyclical graph execution essential for multi-turn educational interactions. The Gemini AI model serves as the cognitive engine within each agent, responsible for generating natural language explanations, constructing quiz questions, and evaluating student answers with semantic understanding.

## 3.2 Data Flow

The request lifecycle follows a deterministic seven-step pipeline:

- 1 User Interaction:** The student selects a task module via the Streamlit sidebar navigation interface (Explain Topic, Generate Quiz, Recall Quiz, or Progress Dashboard).
- 2 API Request:** Streamlit constructs a JSON payload and sends an HTTP POST request to the FastAPI `/process/` endpoint.
- 3 Intent Parsing:** FastAPI delegates the request to `parse_user_input()` in the NLU module, which extracts structured intent, topic, and difficulty parameters.

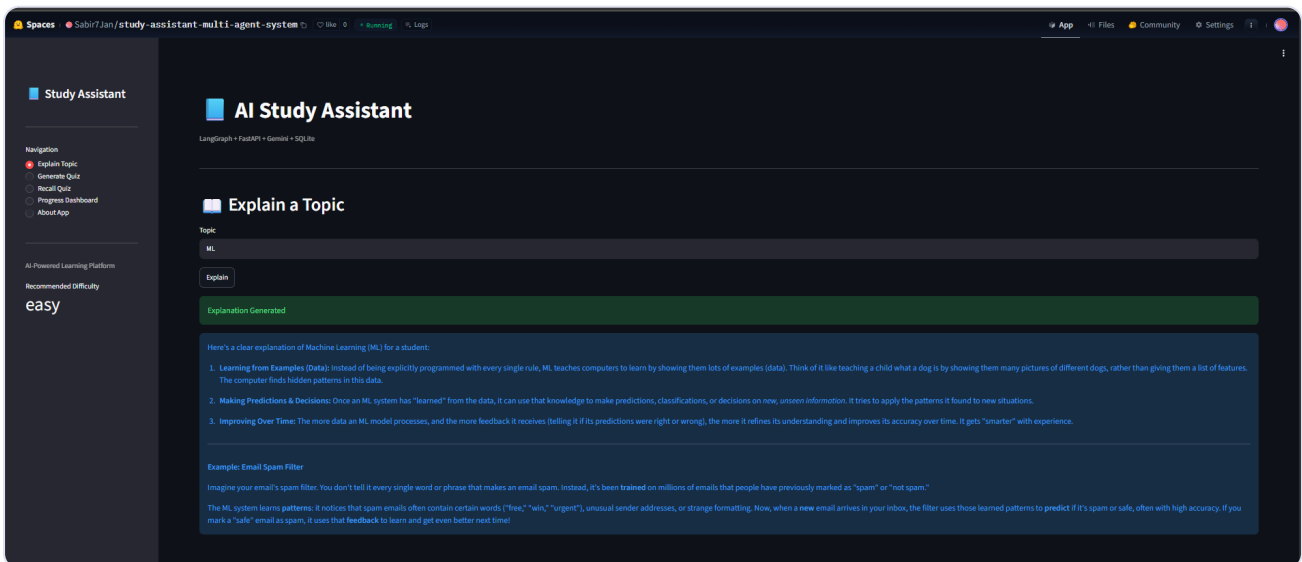
- 4 **Agent** The classified `explain` , `quiz` , `evaluate` , `recall` , `full` .  
**Routing:** intent is or  
routed to the  
corresponding  
LangGraph  
agent graph:
- 5 **LLM** The selected agent constructs a contextually rich prompt and invokes  
**Execution:** Gemini AI to generate educational content, assessment questions, or  
evaluation feedback.
- 6 **Data** Quiz results, scores, and performance `database.py` module for  
**Persistence:** metadata are committed to SQLite via longitudinal  
the tracking.
- 7 **Response** Results are returned to Streamlit as JSON and rendered through  
**Rendering:** conditional UI components including expanders, metric cards, and line  
charts.

**TABLE 1: TECHNOLOGY STACK OVERVIEW**

| Technology | Purpose                                                 | Selection Rationale                                                                                                                                                                                                                                     |
|------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LangGraph  | Multi-agent orchestration and workflow graph definition | Provides stateful, cyclical graph execution essential for multi-turn educational conversations. Supports conditional routing, memory passing between nodes, and human-in-the-loop patterns required for interactive tutoring — unlike linear pipelines. |
| FastAPI    | RESTful backend API layer                               | Async-native architecture, automatic OpenAPI documentation generation, and Pydantic request validation make it ideal for serving ML model endpoints. Performance exceeds Flask and Django for I/O-bound LLM inference workloads.                        |
| Streamlit  | Frontend user interface                                 | Enables rapid prototyping of data-centric applications without JavaScript or HTML. Tight integration with Python data workflows and native charting support makes it optimal for ML demonstration interfaces.                                           |
| Gemini AI  | Large Language Model engine                             | Strong performance in educational content generation, code explanation, and structured output parsing. Context window accommodates multi-question quiz generation and detailed answer evaluation in single inference calls.                             |
| SQLite     | Persistent memory and score storage                     | Zero configuration, serverless operation within the container, and ACID-compliant transactions for quiz result persistence. Lightweight nature is ideal for single-tenant educational applications with moderate data volumes.                          |
| Docker     | Containerized deployment                                | Ensures environment consistency across development and production, encapsulating all Python dependencies, the SQLite database, and the multi-process startup configuration in a single reproducible artifact.                                           |

## 5.1 Explain Topic

The Explain Topic feature allows users to input any academic topic and receive a structured, student-friendly explanation complete with examples and analogies. When a user enters a topic and clicks the Explain button, Streamlit sends a POST request to the `/process/` endpoint with intent set to `explain`. The NLU module parses the topic, and the `explain` LangGraph agent is invoked. This agent constructs a system prompt instructing Gemini to generate a pedagogically sound explanation appropriate for the specified difficulty level. The response is rendered in a formatted container with syntax highlighting for any code examples.

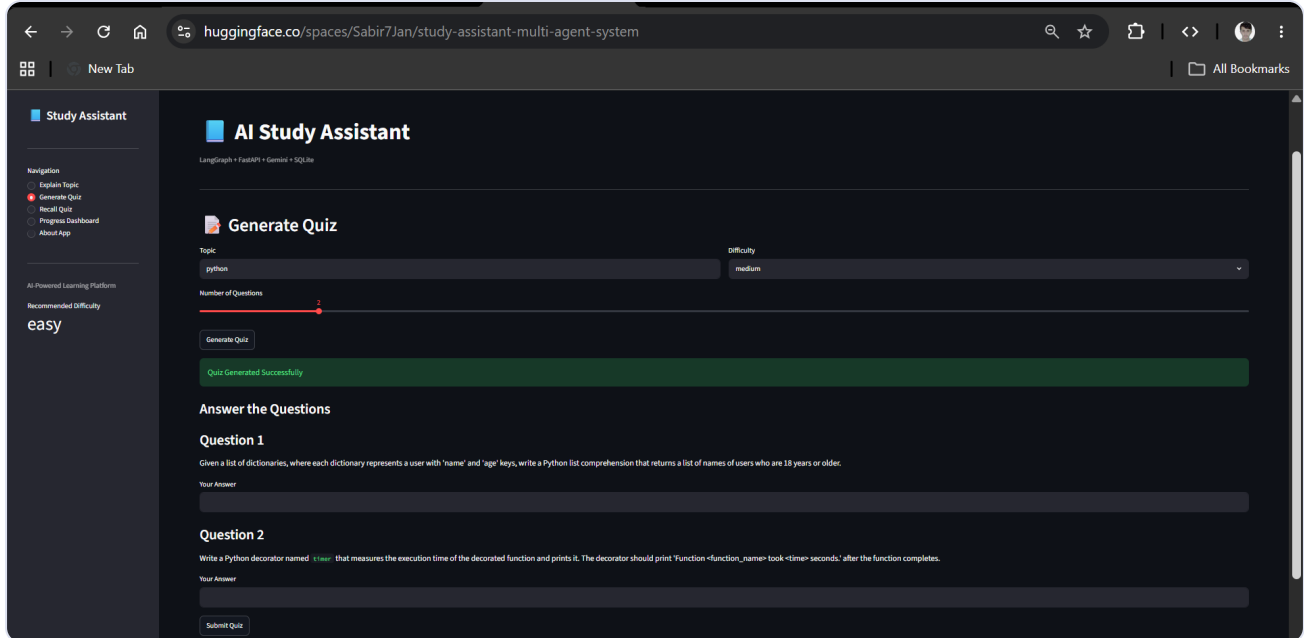


**Figure 1:** *Explain Topic* interface showing a generated explanation for Machine Learning with structured breakdown, real-world analogies, and code examples. The adaptive difficulty indicator is shown in the sidebar.

## 5.2 Generate Quiz

The Generate Quiz feature provides fully configurable assessment generation. Users specify a topic, select difficulty (easy, medium, or hard) from a dropdown selector, and choose the number of questions (1-10) via a slider widget. Streamlit dispatches a `quiz` intent request to FastAPI, which routes to the `quiz` LangGraph agent. This agent constructs a prompt specifying topic, difficulty tier, and question count, then invokes Gemini to generate contextually appropriate open-ended or coding questions. The generated quiz is displayed as a sequence of numbered questions with answer input fields.





**Figure 2:** Generate Quiz interface showing configurable quiz parameters and auto-generated Python programming questions at medium difficulty, including list comprehension and decorator pattern tasks.

## 5.3 Submit and Evaluate Quiz

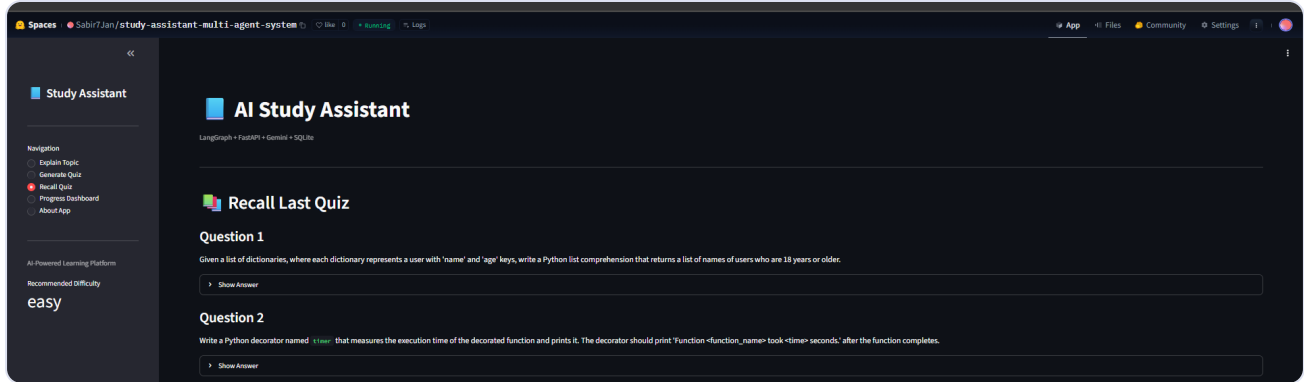
Upon quiz submission, the `evaluate` LangGraph agent receives both the original questions and the student's answers. The agent constructs an evaluation prompt that instructs Gemini to assess each answer for correctness, assign a numerical score, compute an overall percentage, and generate personalized improvement recommendations. The system implements a three-tier performance classification:

- ▶ **≥ 80%** — Excellent Performance
- ▶ **50-79%** — Good Progress
- ▶ **< 50%** — Needs Improvement

Results are persisted to SQLite and immediately returned to the user with a detailed breakdown of correct and incorrect responses along with an actionable recommendation.

## 5.4 Recall Last Quiz

The Recall Last Quiz feature leverages SQLite persistence to retrieve the most recently completed quiz. When activated, the `recall` LangGraph agent queries the database for the latest quiz record, then renders both questions and answers in collapsible Streamlit expander components labeled "Show Answer." This feature enables students to review previous assessments for spaced repetition without regenerating content. The recall agent maintains conversation state through LangGraph's checkpointing mechanism, ensuring that multi-turn review sessions preserve context.



**Figure 3:** Recall Last Quiz interface displaying previously completed Python quiz questions with collapsible answer reveal panels, including code-based questions on list operations and string functions.

## 5.5 Progress Dashboard

The Progress Dashboard provides quantitative insight into learning trajectories. The `/dashboard` endpoint queries SQLite to compute three aggregate metrics: total quizzes taken, average score percentage across all attempts, and a chronological score history array. The frontend renders these metrics using Streamlit's native `st.line_chart` component, producing a time-series visualization of performance trends. The dashboard enables students to identify improvement patterns, detect knowledge gaps across topics, and monitor their overall learning velocity.



**Figure 4:** Progress Dashboard displaying total quizzes (5), average score (26.67%), recommended difficulty (easy), and a score history line chart across all attempts.

## 5.6 Adaptive Difficulty Recommendation

The Adaptive Difficulty Recommendation engine operates as a background analytics service accessed via the `/adaptive` endpoint. The `get_recommended_difficulty()` function in `database.py` analyzes the user's recent quiz history, computing a rolling accuracy score over the last five attempts:

- Average accuracy > **75%** → Promote to higher difficulty tier

- ▶ Average accuracy < **40%** → Reduce difficulty to prevent discouragement
- ▶ **Intermediate scores** → Maintain current difficulty level

This recommendation is displayed prominently in the sidebar as a persistent metric, providing real-time guidance on optimal challenge calibration throughout the learning session.

The backend exposes a RESTful API through FastAPI with comprehensive endpoint coverage for all system operations. The API is automatically documented via OpenAPI at the [/docs](#) route. Table 2 documents the complete endpoint specification.

**TABLE 2: REST API ENDPOINT SPECIFICATION**

| Endpoint                   | Method | Request Format                                                   | Response Format                                               | Description                                                                             |
|----------------------------|--------|------------------------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <a href="#">/process/</a>  | POST   | { "user_input": "...", "difficulty": "...", "num_questions": N } | { "intent": "...", "result": {...} }                          | Main routing endpoint — handles all intent types: explain, quiz, evaluate, recall, full |
| <a href="#">/dashboard</a> | GET    | None                                                             | { "total_quizzes": N, "average_score": N.N, "scores": [...] } | Returns aggregate performance metrics and chronological score data for visualization    |
| <a href="#">/adaptive</a>  | GET    | None                                                             | { "recommended_difficulty": "easy medium hard" }              | Computes optimal difficulty level based on rolling accuracy of recent quiz attempts     |
| <a href="#">/memory</a>    | GET    | None                                                             | { "chat_history": [...] }                                     | Retrieves LangGraph conversation memory for context-aware multi-turn interactions       |
| <a href="#">/</a>          | GET    | None                                                             | { "message": "Running" }                                      | Health check endpoint for container orchestration and uptime monitoring                 |

The application is containerized using Docker and deployed on Hugging Face Spaces. The `Dockerfile` specifies a Python 3.9+ base image, installs dependencies from `requirements.txt`, copies the application source code, and exposes the necessary ports for both the FastAPI backend and Streamlit frontend.

The `start.sh` entrypoint script initializes the multi-process environment: it starts the FastAPI server in the background, then launches the Streamlit frontend process. This concurrent startup pattern ensures both services are available within a single container, simplifying deployment topology and reducing infrastructure overhead.

**Hugging Face Spaces** provides the hosting infrastructure with automatic HTTPS termination, persistent volume mounting for the SQLite database, and public URL generation. The Docker-compatible runtime enables seamless migration from local development to production without environment drift.

The live deployment is accessible at:

<https://huggingface.co/spaces/Sabir7Jan/study-assistant-multi-agent-system>

The repository source files are available at:

<https://huggingface.co/spaces/Sabir7Jan/study-assistant-multi-agent-system/tree/main>

TABLE 3: TECHNICAL CHALLENGES AND RESOLUTION STRATEGIES

## Multi-Agent Intent Routing

NLU Engineering

Designing a robust routing mechanism that correctly classifies user intent and dispatches to the appropriate LangGraph graph required careful prompt engineering in the NLU module. The solution implements a structured output parser that extracts intent, topic, and difficulty with high reliability, combined with fallback logic for ambiguous inputs that defaults to the `explain` intent — ensuring graceful degradation without error exposure to the user.

## Persistent Memory Across Sessions

State Management

LangGraph's default in-memory checkpointer loses state on container restart. The solution integrates SQLite as a persistent checkpoint store via `database.py`, ensuring quiz history, conversation threads, and performance metrics survive deployment updates and container restarts. The module implements connection pooling and atomic transactions to prevent data corruption under concurrent request conditions.

## Adaptive Difficulty Calibration

Algorithm Design

Implementing a difficulty recommendation algorithm that responds appropriately to performance volatility required designing a rolling-window accuracy metric rather than a naive cumulative average. The algorithm examines the last five attempts, applies tiered thresholds (75% for promotion, 40% for demotion), and debounces rapid difficulty oscillations through hysteresis logic — preventing jarring back-and-forth difficulty changes during unstable performance periods.

## Results and Evaluation

---

The AI Study Assistant demonstrates three core capabilities that validate its architectural design:

- 1 The **adaptive learning pipeline** successfully generates contextually appropriate educational content across diverse topics including computer science, mathematics, and natural sciences. The NLU module achieves reliable intent classification, and the LangGraph routing system correctly dispatches requests to specialized agent graphs with minimal latency.
- 2 The **automatic evaluation system** produces nuanced grading that extends beyond binary correctness. Gemini's evaluation prompt is engineered to identify partially correct answers, award proportional credit, and generate actionable improvement recommendations. The three-tier performance classification provides students with clear, motivating feedback.
- 3 The **persistent memory architecture** successfully retains quiz history across sessions, enabling longitudinal performance tracking. The Progress Dashboard renders accurate trend visualizations, and the adaptive difficulty recommendation engine responds appropriately to changes in student performance over time.



The following enhancements are planned for subsequent development iterations:

## User Authentication and Multi-User Support

Implementing session-based authentication with unique user profiles will enable multiple learners to use the system concurrently with isolated performance histories. This requires extending the database schema with a `users` table and adding JWT-based authentication middleware to FastAPI endpoints.

## Spaced Repetition Algorithm

Integrating a SuperMemo-2 or SM-15 spaced repetition scheduler would optimize quiz recall timing based on each question's historical difficulty for the individual learner. Questions answered incorrectly would reappear at scientifically calibrated intervals to maximize long-term retention.

## Voice Input and Multimodal Interaction

Adding speech-to-text input via the Web Speech API or Whisper model would enable hands-free quiz participation and verbal topic queries. Text-to-speech output for explanations would further enhance accessibility for visually impaired learners.

## Multi-LLM Provider Support

Abstracting the LLM client through a provider interface pattern would enable runtime switching between Gemini, GPT-4, Claude, and open-source models. This increases deployment flexibility, reduces vendor lock-in, and allows cost-performance optimization based on workload characteristics.

The AI Study Assistant represents a complete full-stack implementation of a multi-agent intelligent tutoring system, demonstrating proficiency in modern AI engineering practices including LLM integration, graph-based agent orchestration, RESTful API design, and containerized cloud deployment. The project successfully addresses the core limitations of static self-study tools by delivering adaptive, personalized learning experiences through a clean architectural separation of concerns.

The LangGraph multi-agent framework proved highly effective for educational applications requiring stateful, context-aware interactions. The persistent SQLite storage layer and adaptive difficulty recommendation engine combine to create a learning system that genuinely improves with usage — calibrating challenge levels to individual proficiency trajectories. This project serves as a strong portfolio piece showcasing the intersection of machine learning, software engineering, and educational technology design.

- [1] LangChain. (2024). *LangGraph: Build resilient language learning agents as graphs*.  
<https://python.langchain.com/docs/langgraph>
- [2] FastAPI. (2024). *FastAPI framework documentation*.  
<https://fastapi.tiangolo.com>
- [3] Streamlit. (2024). *Streamlit documentation: Create Python data apps*.  
<https://docs.streamlit.io>
- [4] Google. (2024). *Gemini API developer documentation*.  
<https://ai.google.dev/docs>
- [5] SQLite Consortium. (2024). *SQLite: Small. Fast. Reliable*.  
<https://sqlite.org>
- [6] Docker Inc. (2024). *Docker documentation: Containerize applications*.  
<https://docs.docker.com>
- [7] Hugging Face. (2024). *Spaces: Host ML demo apps*.  
<https://huggingface.co/docs/hub/spaces>
- [8] Google. (2024). *Generative AI on Vertex AI documentation*.  
<https://cloud.google.com/vertex-ai/docs/generative-ai>

## Technologies Used

LangGraph

FastAPI

Streamlit

Gemini AI

SQLite

Docker

Hugging Face Spaces

Python 3.9+

Pydantic

Pandas

REST API

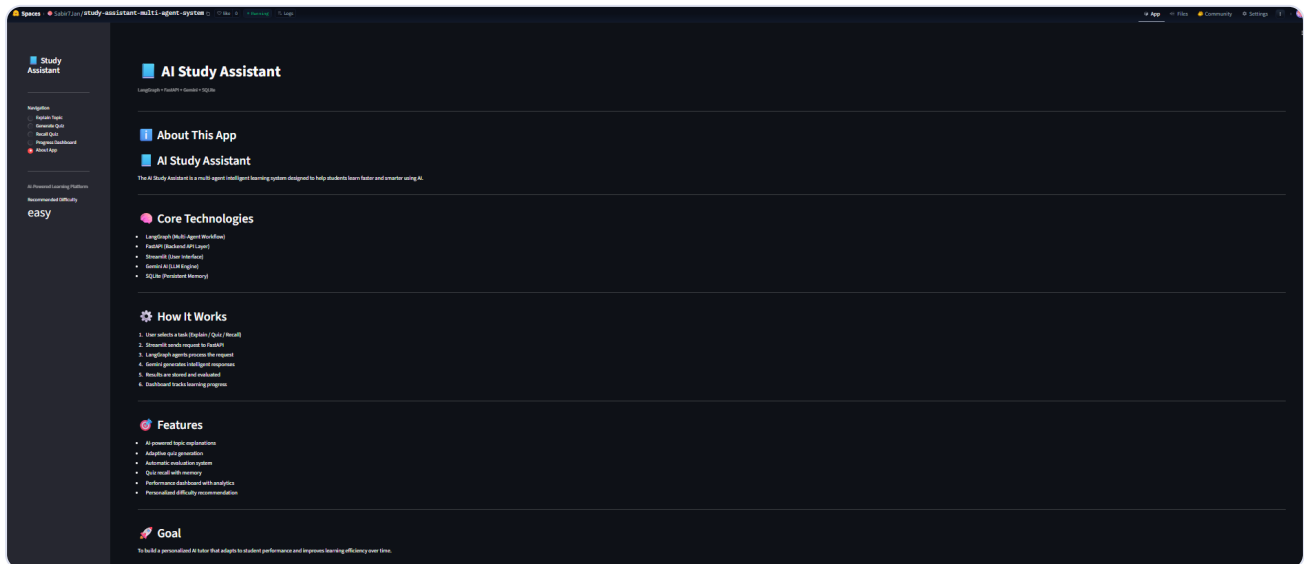
CORS Middleware

NLU / Intent Parsing

Adaptive ML

# Appendix: Application Screenshots

The following figures reference the application's production interface as deployed on Hugging Face Spaces. Screenshots were captured from the live deployment at [huggingface.co/spaces/Sabir7Jan/study-assistant-multi-agent-system](https://huggingface.co/spaces/Sabir7Jan/study-assistant-multi-agent-system).



**Figure 5:** About This App page displaying core technologies (LangGraph, FastAPI, Streamlit, Gemini AI, SQLite), workflow description, feature summary, and project goal statement.